```
LOCAL MODELS IN PRACTICE
running a 4-bit 8B on a real, hard task — and what actually moved
text-to-SQL · execution-verified · 16 GB laptop · MLX
```

A personal field report from hands-on local-model experiments on constrained hardware. One model studied deeply (Qwen3-8B, 4-bit) on a real, hard task (text-to-SQL), against a frontier baseline, across ~10 different methods. The absolute numbers are one data point; the ***mechanisms and the setup lessons generalize****.*

TL;DR (the quotable part)

1. **For local-model* usability*, scaffolding beats parameters. A 4-bit 8B* with a tool** — let it run its own SQL, see the error/result, and fix it — reached its accuracy ceiling **gold-free in ~2 turns**, beating sampling, voting, and fine-tuning. The highest-leverage decision isn't the model or the quant level; it's **whether the model gets tools and a feedback loop.**
2. **There is a hard capability ceiling, and it belongs to the base model.** On hard, heterogeneous queries the 4-bit 8B topped out at ~40% *across every method tried*. Raising that needs a bigger/better base model or RL — not prompt or scaffold tricks.
3. **Accuracy is not one number — it depends on task* shape.** On homogeneous tasks (one skill, many instances) a local model is strong and can even self-improve to ~98%. On heterogeneous tasks (every query different) it is capability-bound. Any eval reporting a single accuracy hides this.
4. **4-bit quantization is the practical small-RAM lever.** Everything ran in 4-bit at ~5–6 GB resident on a 16 GB machine. Weight-*streaming* from SSD is the wrong lever for interactive/tool use (latency) — quantize instead.

Setup (what I actually ran)

- **Hardware:** Apple Silicon laptop, **16 GB unified memory** — deliberately small; a good proxy for the

"limited RAM" question.

- **Runtime:** MLX (Apple's local inference stack). One model resident at a time.
- **Local model:** `Qwen3-8B`, **4-bit** quantized (`mlx-community/Qwen3-8B-4bit`), ~5–6 GB resident.
- **Frontier baseline:** a frontier model (Claude) via CLI — used as a top-of-line reference and as a

"teacher" to generate correct examples for fine-tuning experiments.

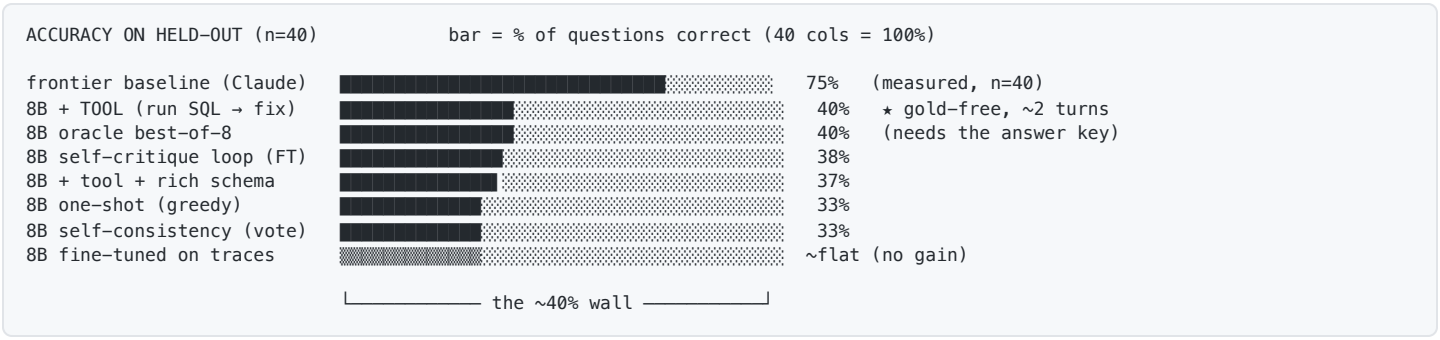
- **Task: BIRD** mini-dev, **text-to-SQL** — natural-language question + database schema → SQL. Real

SQLite databases. Used the *simple+moderate* difficulty slice; held-out set of **40** questions, fixed across every method for apples-to-apples comparison.

- **Eval function (the core of it): execution accuracy** — run the model's SQL *and* the gold SQL

against the real database and compare result sets. Free, deterministic, fully automated. No human grading, no LLM-as-judge.

Results at a glance



The whole story in one picture: every lever — sampling, voting, fine-tuning, tools, schema enrichment — lands at **~40%**. Tool-use is the only one that gets there *cheaply and without the answer key*.

Every method I tried (the full list)

#	Method	Idea	Result (n=40)
1	One-shot (greedy)	single attempt, plain prompt	13/40 (33%)
2	Self-consistency	8 samples, majority result wins	13/40 (33%)
3	Oracle best-of-8	8 samples, keep if *any* is correct (upper bound)	16/40 (40%)
4	pass@k curve	how the oracle bound grows with k	plateaus at 40%
5	**Agentic tool-use**	run the SQL, read error/result, fix; gold-free stop	**16/40 (40%) ★**
6	Self-distillation	fine-tune on the model's own verified-correct traces	flat
7	Teacher-bootstrap distill	fine-tune on 50 *frontier-correct* traces	flat
8	Teacher-bootstrap, scaled	same, scaled to ~100 correct traces	flat
9	Factored / decomposed distill	teach it to break a query into sub-skills, then compose	flat
10	Self-critique loop (FT)	fine-tune the DRAFT→CHECK→FINAL loop into the weights	15/40 (~38%)
11	Tool + rich schema	give sample values per column (value-format knowledge)	15/40 (37%)

Methods 6–9 are the "can a small model fine-tune itself good at this?" line of attack. **They all stayed flat** — even on correct frontier-authored examples — because ~50–100 examples can't cover a heterogeneous domain, and the bottleneck is generation capability, not training signal.

What the model actually gets wrong — and how the tool fixes it

Real failure mode (representative). The schema hides a foreign key the model forgets to join:

```
-- SCHEMA (compact form given to the model)
superhero(id, superhero_name, full_name, gender_id, race_id, publisher_id, ...)
gender(id, gender)

-- QUESTION : "How many superheroes are female?"
-- HINT     : female refers to gender = 'Female'

-- 8B one-shot, turn 1                                          ✗ WRONG
SELECT COUNT(*) FROM superhero WHERE gender = 'Female';
   └─► sqlite error: no such column: gender

-- 8B with the run_sql tool, turn 2 (after seeing the error)    ✓ CORRECT
SELECT COUNT(*)
FROM   superhero AS T1
JOIN   gender    AS T2 ON T1.gender_id = T2.id
WHERE  T2.gender = 'Female';
   └─► runs, result matches gold
```

That single feedback step — "*no such column: gender*" — is information the model didn't have at generation time. That is the whole reason tool-use works where blind voting doesn't.

The internalized version (method 10) teaches the model to do that critique *itself*, without the tool:

```
DRAFT:  SELECT COUNT(*) FROM superhero WHERE gender = 'Female';
CHECK:  'gender' is not a column on superhero — it's gender_id, a foreign key into
        gender(id, gender). This needs a join.
FINAL:  SELECT COUNT(*) FROM superhero T1
        JOIN gender T2 ON T1.gender_id = T2.id
        WHERE T2.gender = 'Female';
```

It reaches ~38% tool-free — the loop *is* partly learnable into the weights — but still can't beat the ~40% ceiling.

Why richer schema (method 11) didn't help. Sample values were injected inline so the model could see the value formats:

```
plain schema:   gender(id, gender)
rich schema:    gender(id, gender[Male,Female])      ← sample distinct values inline
```

It fixed *some* value-format mistakes but the overall number didn't move — schema ignorance wasn't the binding constraint; raw generation capability was.

The capability ceiling, seen as a curve

```
pass@k  —  oracle "any-of-k samples correct"  (upper bound with a perfect selector)

 45% |
 40% |                  • — • — • — •      ← PLATEAU: more samples add nothing
 35% |          •
 30% |     •
 27% |  •
     +----+----+----+----+----+----+----+
        1    2    4    6    8   12   16        k = number of samples
```

Past k=8 the bound stops rising: on ~60% of hard queries the correct SQL **never appears in any sample**. That is a generation-capability limit, not a selection or sampling limit. No local trick conjures an answer the model can't produce.

The five findings, with the "why"

1. Tools > parameters for usability. Execution feedback is *real information the model lacks*. Plain voting can't replicate it because the model's wrong answers don't agree on one wrong result, so the majority is often wrong. Budget for an agent loop + a verifier, not just a bigger model.

2. The ~40% ceiling is base capability, confirmed six ways (sampling, voting, two kinds of fine-tuning, tool-use, schema enrichment all cap there). Scaffolding sets how *close* you get to the ceiling; it doesn't raise it. Raising it needs a stronger base model, far larger fine-tuning corpora (published SQL wins use ~900k synthetic examples), or RL.

3. Task* shape *dominates the number. Same model, same effort: ~98% on a homogeneous self-taught skill, ~40% on heterogeneous BIRD. A single "accuracy" per model is misleading without a task-shape axis.

**4. Locally you can teach a* loop*, not broad* coverage*. Fine-tuning on correct traces (even frontier-correct) left heterogeneous accuracy flat; what transferred was the self-critique* behavior* (~38%). You can cheaply install a reusable habit; you can't cheaply install domain breadth.

5. Quantization is the right small-RAM lever; streaming is not. 4-bit Qwen3-8B ran fine at ~5–6 GB and was capability-bound, not quant-bound, on this task. SSD weight-streaming adds per-token latency that kills an interactive tool loop — and the loop is where the value is.

Practical gotchas (the time-savers)

```
⚠  Eval truncation = false failures.
   If eval max_tokens < the model's answer length, the answer is silently chopped
   and scored 0. This produced a "total failure" that was actually 98%. Always
   diagnose model-vs-eval before believing a bad score.

⚠  One model at a time on small RAM.
   16 GB holds one ~8B comfortably; don't co-load. Fine-tuning needs aggressive
   settings (batch 1, fewer layers, short sequences, gradient checkpointing) or
   it OOMs on long inputs.

⚠  Compact the prompt.
   table(col, col, …) instead of full CREATE DDL was ~1.5× faster, no accuracy loss.
   Prompt size is a real per-call cost.

⚠  Progress bars corrupt output.
   Library download/progress bars merge into stdout and break parsed results —
   disable them in batch runs.

✓  Use a free, deterministic verifier when one exists (execution, unit tests, regex).
   It removes humans and LLM-judges from the loop and makes the eval trustworthy.
```

Honest caveats

- **One model, one benchmark, one difficulty slice, n=40.** The ~40% number is specific to Qwen3-8B-4bit on

hard BIRD. The *mechanisms* (tools reach the ceiling, the ceiling is base capability, shape matters, quant works) are what generalize.

- This is **one deep data point**, not a model leaderboard. A matrix across other open-weight models is the

obvious next step — the harness is model-agnostic, so it's mostly config.

Recommendations

1. **Evaluate by task complexity — and add two axes:** *task homogeneity* (homogeneous vs heterogeneous) and *with-tools vs without-tools*. They explain more variance than model choice.
2. **Make "agent loop + free verifier" a first-class condition**, not an afterthought. It's the difference between a leaderboard and a useful conclusion.
3. **Quantize, don't stream.** Test 8-bit and 4-bit; expect 4-bit to be the practical floor. Treat SSD-streaming as a curiosity, not an interactive-deployment path.

4. **The headline is "scaffolding beats parameters for local usability."** Non-obvious, defensible, and far more interesting than "model X scored Y%".
5. **Build the harness once.** An execution-verified, difficulty-sliced harness with a frontier baseline and a 4-bit setup pays for itself; adding more candidate models is then a small delta.

Next step would be extending the harness to other open-weight models (e.g. GLM-5.2 / Gemma / DeepSeek) to produce a full model × quant × (with/without tools) matrix.